

CellQuantX

Automated Cell Detection & Counting from Microscopy Images

Technical Documentation & Feature Report

Platform: Streamlit Web Application

Version: 1.0 · April 2026

BHUVANESHWARAN G — 23BBT012

Computer Vision · Streamlit · OpenCV · Google Gemini AI ·

Contents

- 1 **Executive Summary**
- 2 System Requirements
 - 2.1 Software Dependencies
 - 2.2 Installation
- 3 Application Architecture
- 4 User Interface
- 5 Image Processing Pipeline
- 6 Segmentation Methods
- 7 Analysis Results — Sample Run
 - 7.1 Image Analysis Results
 - 7.2 Area Distribution Histogram
 - 7.3 Threshold Sensitivity Analysis
 - 7.4 Morphology: Circularity vs Area
 - 7.5 Density Heatmap Overlay
 - 7.6 Phenotype Summary
 - 7.7 Intensity Distribution
- 8 Output Statistics
- 9 Expert Features
- 10 Export Options
- 11 Quick Start & Workflow
- 12 Troubleshooting
- 13 Limitations & Future Work

1. Executive Summary

Overview

CellQuantX is a browser-based, Python-powered cell counting and analysis application that enables rapid, reproducible quantification of cells in microscopy images. Users upload cell images through an intuitive Streamlit interface, tune detection parameters, and receive annotated result images alongside a full statistical report — all within seconds.

The application combines a Streamlit web frontend with a multi-stage computer vision pipeline built on OpenCV. The pipeline applies CLAHE contrast enhancement, Gaussian denoising, multi-mode thresholding, morphological cleanup, and a watershed-based cell separation algorithm to accurately count individual cells even in dense, clustered populations.

Key Highlights

- Four segmentation algorithms: Global Threshold, Watershed, Adaptive Threshold, Enhanced Labeling Mode
- AI-powered biological interpretation via Google Gemini (6 analysis modes)
- Expert features: Spatial Density Heatmap, Phenotypic Clustering, Intensity Quantification
- Batch processing support for multiple microscopy images
- Export to CSV, annotated PNG, and full PDF report
- Calibrated physical measurements ($\mu\text{m}/\text{pixel}$, cells/mm^2 , confluence %)
- Method comparison with critical analysis across up to 3 segmentation algorithms
- Quick preset profiles for common microscopy scenarios

2. System Requirements

2.1 Software Dependencies

Package	Min Version	Role	Key Functions Used
streamlit	1.28.0	Web framework	UI widgets, session state, caching, file upload
opencv-python	4.5.0	Computer vision	CLAHE, Gaussian blur, watershed, contours, distance transform
numpy	1.21.0	Array operations	Image arrays, masking, statistics, pixel math
pandas	1.3.0	Data analysis	Cell data tables, CSV export, batch summaries
matplotlib	3.4.0	Plotting	Histograms, scatter plots, sensitivity graphs
Pillow	8.0.0	Image I/O	Image loading, format conversion, PNG saving
reportlab	3.6.0	PDF generation	PDF report with tables, images, styled text

Package	Min Version	Role	Key Functions Used
google-generativeai	0.3.0	AI interpretation	Gemini model calls for biological analysis

2.2 Installation

```
# Step 1: Install all dependencies
pip install streamlit>=1.28.0 opencv-python>=4.5.0 numpy>=1.21.0 \
    pandas>=1.3.0 matplotlib>=3.4.0 Pillow>=8.0.0 \
    reportlab>=3.6.0 google-generativeai>=0.3.0 python-dotenv>=0.19.0

# Step 2: Configure Gemini API key (optional, for AI features)
echo GEMINI_API_KEY=your_key_here > .env

# Step 3: Launch the application
streamlit run app.py

# Step 4: Open browser at http://localhost:8501
```

3. Application Architecture

CellQuantX follows a modular design with Streamlit as the frontend, and OpenCV/analysis modules as the computation layer. The application is split into several Python modules for maintainability.

```
cellquantx/
  app.py           # Main Streamlit app (UI + orchestration)
  segmentation.py  # preprocess() and segment() functions
  analysis.py      # generate_heatmap() and count_cells()
  utils.py         # ensure_rgb() and save_figure_bytes()
  ai_module.py     # get_ai_interpretation() via Gemini
  .env             # GEMINI_API_KEY (optional)
  requirements.txt # Python dependencies
```

The Streamlit reactive flow:

1. User uploads image(s) via drag-and-drop or file picker
2. Sidebar controls configure segmentation method and parameters
3. Preprocessing pipeline runs (cached with `@st.cache_data`)
4. Segmentation algorithm produces binary mask
5. Cell counting and metrics are computed from detected contours
6. Optional expert features (heatmap, phenotype, intensity) run if enabled
7. AI interpretation is generated on demand via Gemini API
8. Results are displayed and available for export (CSV, PNG, PDF)

4. User Interface

CellQuantX uses Streamlit's reactive component model with a custom dark-theme scientific aesthetic. The interface is divided into a collapsible left sidebar for controls and a main panel for results.

4.1 Main Application Screen

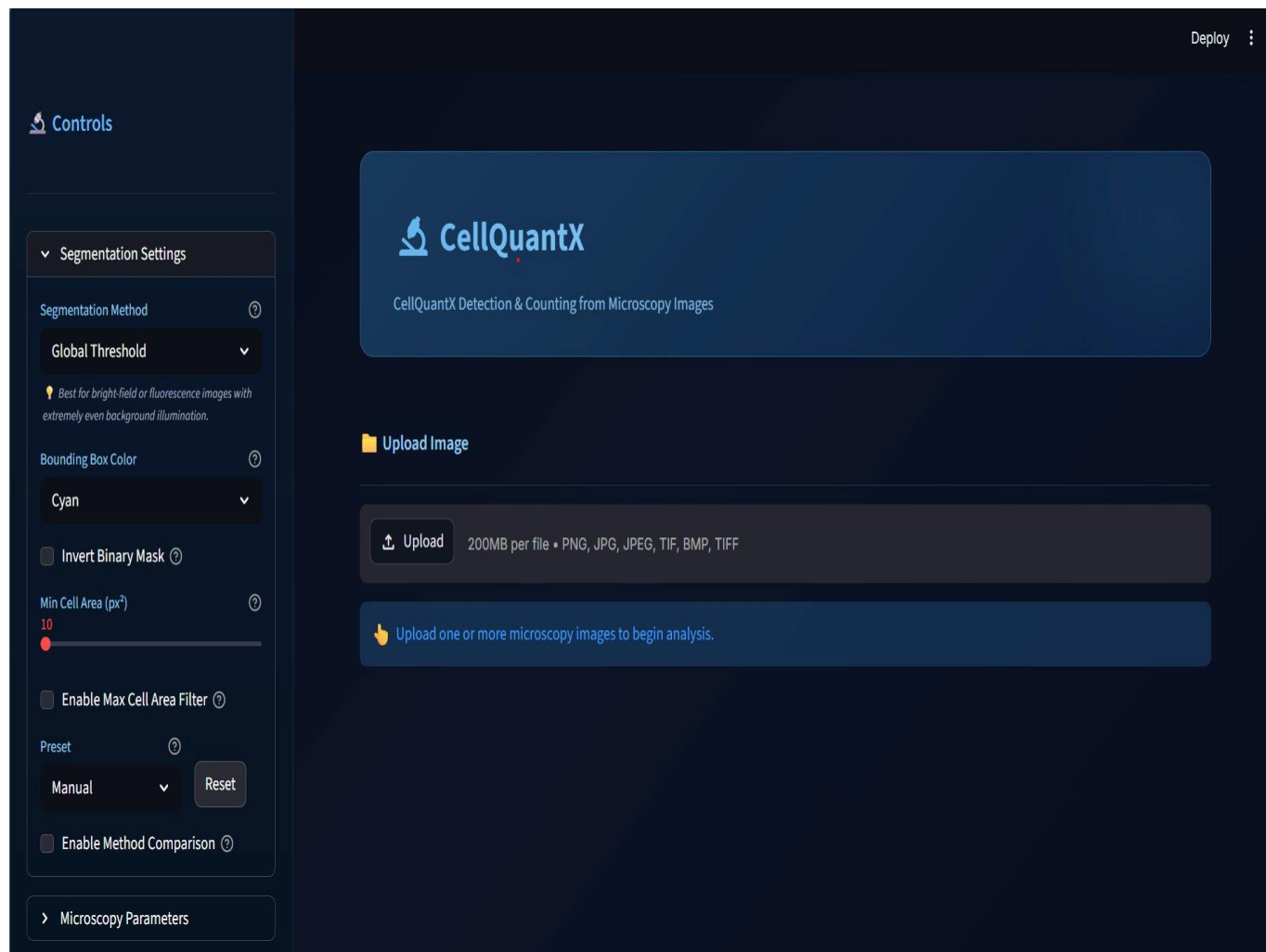


Figure 1. CellQuantX main interface showing the upload zone and sidebar controls with Segmentation Settings expanded.

4.2 Sidebar Controls

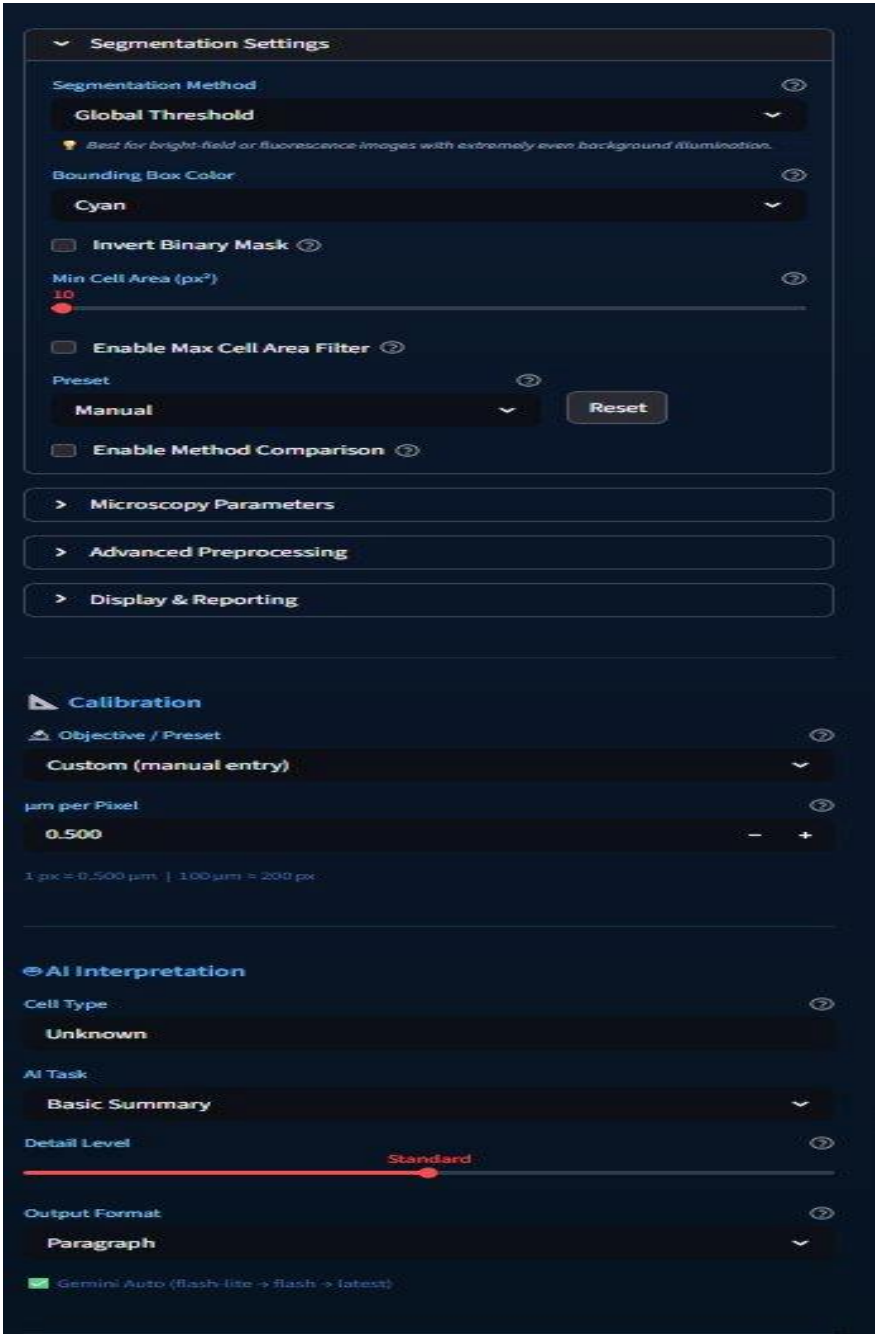


Figure 2. Full sidebar panel showing Segmentation Settings, Calibration, and AI Interpretation controls.

4.3 Preprocessing Stages View

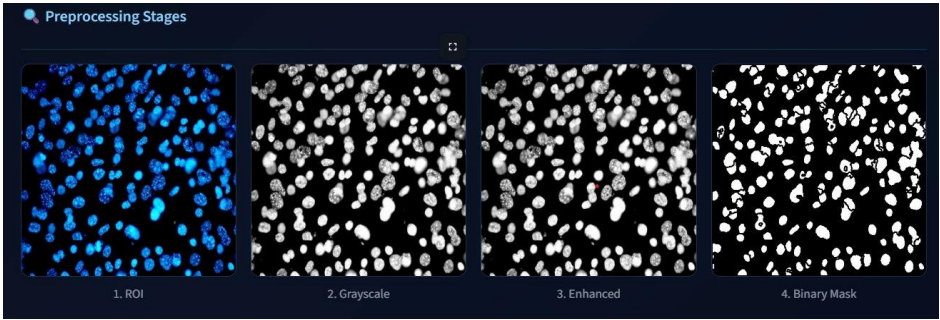


Figure 3. Four-stage preprocessing pipeline visualization: ROI → Grayscale → Enhanced → Binary Mask (DAPI-stained fluorescence nuclei).

4.4 Controls Reference

Control	Type	Range / Options	Effect
Segmentation Method	Dropdown	4 options	Switches detection algorithm
Bounding Box Color	Dropdown	5 colors	Annotation color on output image
Invert Binary Mask	Checkbox	On/Off	Flips foreground/background polarity
Min Cell Area (px²)	Slider	5–2000	Filters debris; lower detects smaller cells
Preset Profile	Dropdown	6 presets	One-click setup for common scenarios
Watershed Strictness	Slider	0.1–0.9	Controls seed peak threshold for cell splitting
Min Circularity	Slider	0.0–1.0	Removes elongated / fragmented objects
µm per Pixel	Number input	0.001–20.0	Microscope calibration for physical units
Objective Preset	Dropdown	17 options	Auto-fills µm/pixel from common objectives

5. Image Processing Pipeline

The core of CellQuantX is a multi-stage computer vision pipeline. Each stage transforms the image to progressively isolate individual cells.

5.1 Stage 1 — Color Channel Selection & Flat-Field Correction

For multi-channel (RGB) images, the user selects which channel to analyze: Red, Green, Blue, Grayscale, or Auto-Detect (brightest by mean intensity). Optional flat-field correction removes vignetting by dividing each pixel by a Gaussian-blurred version of the image.

5.2 Stage 2 — Contrast Enhancement (CLAHE)

CLAHE (Contrast Limited Adaptive Histogram Equalization) divides the image into small tiles and equalizes each independently, preventing over-amplification of noise in uniform regions and improving visibility of faint cells.

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
enhanced = clahe.apply(blur)
```

5.3 Stage 3 — Gaussian Blur

A Gaussian filter suppresses high-frequency pixel noise before thresholding. The kernel size is user-adjustable via the Pre-Blur Radius slider (default 3 px, range 1–15).

5.4 Stage 4 — Multi-Mode Thresholding

Four thresholding strategies are available. The default Global Threshold uses a hybrid Otsu-Triangle approach (80% Triangle weight) for maximum sensitivity to faint cells.

5.5 Stage 5 — Morphological Cleanup

Morphological opening removes noise particles; closing fills internal holes in cell bodies. An automatic polarity detector selects normal or inverted mask based on foreground density heuristics.

```
kernel = np.ones((3, 3), np.uint8)
binary = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
binary = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)
```


5.6 Stage 6 — Watershed Segmentation

Watershed separates touching or overlapping cells using distance-transform seeding. Peaks in the distance map become markers; flood-fill creates boundaries between adjacent cells.

```
dist = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
dist_s = cv2.GaussianBlur(dist, (5, 5), 0.5)
local_max = cv2.dilate(dist_s, np.ones((nms_k, nms_k), np.uint8))
sure_fg[(dist_s >= (local_max - 1e-6)) & (dist_s > noise_floor)] = 255
_, markers = cv2.connectedComponents(sure_fg)
markers = cv2.watershed(img_color, markers)
```

6. Segmentation Methods

Method	Mechanism	Strengths	Weaknesses
Global Threshold	Hybrid Otsu-Triangle threshold (80% Triangle) applied globally	Fast, deterministic, no tuning needed	Fails on non-uniform illumination; merges touching cells
Adaptive Threshold	Local threshold per 11x11 px tile; fuses Gaussian + Mean adaptive maps	Robust to illumination gradients; preserves faint cells	Can fragment large cells or over-segment textured backgrounds
Watershed	Distance-map peaks become seeds; flood-fill creates dams between touching cells	Separates overlapping cells; shape-preserving	Heavier compute; over-segments irregular shapes
Enhanced Labeling Mode	Top-Hat background flattening + Triangle threshold at 50% sensitivity	Detects faint/tiny cells; immune to shape breaking contours	May include large artifacts; slower on large images

7. Analysis Results — Sample Run

The following results were generated by running CellQuantX on a DAPI-stained fluorescence microscopy image of nuclei. Settings used: Global Threshold, Min Cell Area = 10 px², Calibration = 0.500 µm/px.

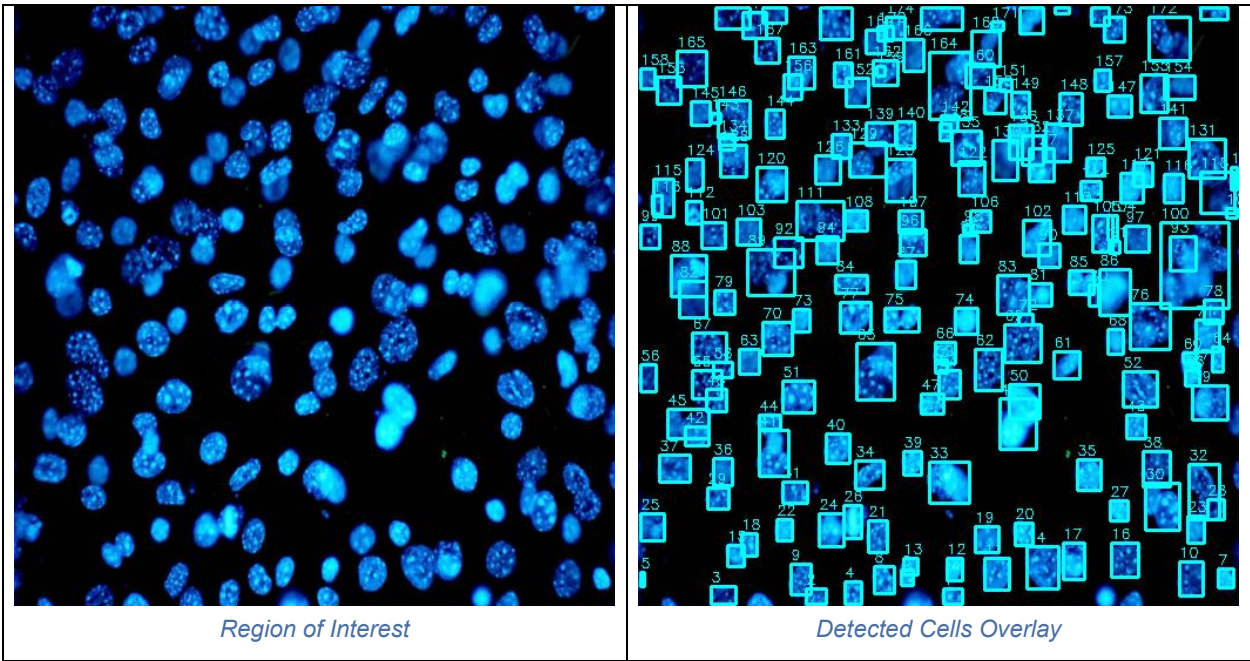
Metric	Value
Image	gettyimages-173543371-612x612.jpg
Cell Type	Unknown
Segmentation Method	Global Threshold

Metric	Value
Calibration (µm/px)	0.500
Total Cells Detected	183
Cell Density (cells/mm²)	2792.36
Confluence / Coverage	22.1%
Mean Cell Area (µm²)	71.55
Mean Circularity	0.6547
Mean Aspect Ratio	0.8961
Mean Solidity	0.8678
Mean Intensity (ADU)	203.09

AI Biological Interpretation: Automated analysis identified 183 cells in the field with high density and a mean area of 71.5 µm². Mean circularity of 0.655 suggests moderately circular morphology, which is consistent with normal physiological conditions. This indicates the sample is well-segmented and may require further review if the morphology appears abnormal.

7.1 Image Analysis Results

Left: Original region of interest (ROI). Right: Detected cells overlay with bounding boxes and cell ID labels.



7.2 Area Distribution Histogram

Distribution of cell areas (μm^2) across the detected population. Mean = $71.5 \mu\text{m}^2$ (yellow dashed), Median = $61.6 \mu\text{m}^2$ (blue dotted). The distribution is right-skewed, indicating most cells are small with a few larger outliers.

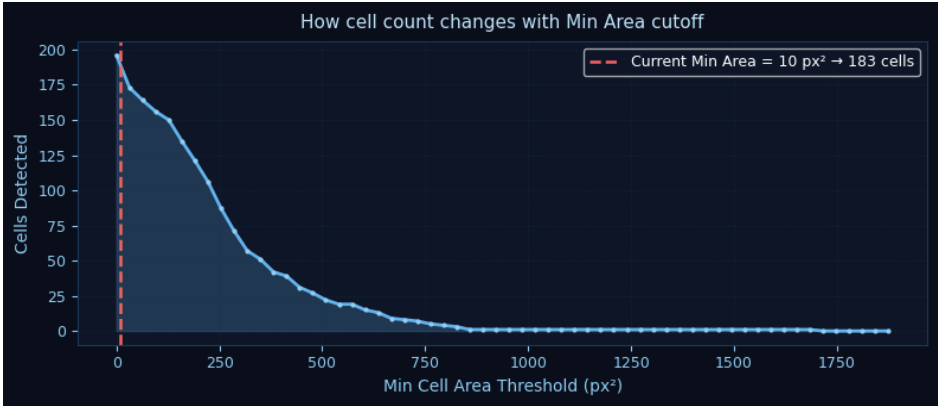


Figure 4. Cell Area Distribution histogram. Viridis-colored bars show frequency per size bin.

7.3 Threshold Sensitivity Analysis

Shows how cell count changes as the minimum area cutoff increases. At the current setting ($10 \text{ px}^2 \rightarrow 183 \text{ cells}$), the curve is in a steep region, meaning small adjustments significantly affect count. Setting the threshold to $\sim 100 \text{ px}^2$ would yield ~ 20 cells.

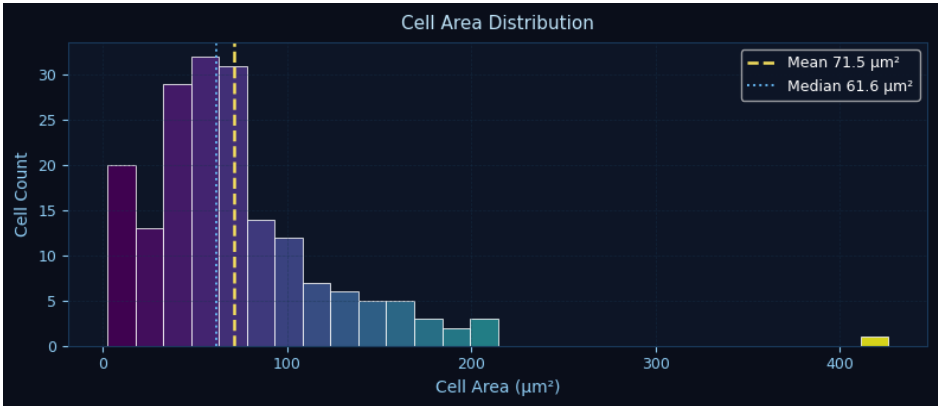


Figure 5. Sensitivity Analysis: Cells Detected vs. Min Area Threshold. Red dashed line marks the current threshold.

7.4 Morphology: Circularity vs Area

Scatter plot of cell circularity against area, coloured by phenotype. The dotted yellow line at circularity = 0.75 separates Compact-Round from other phenotypes. Most cells cluster below 100 μm^2 with moderate circularity.

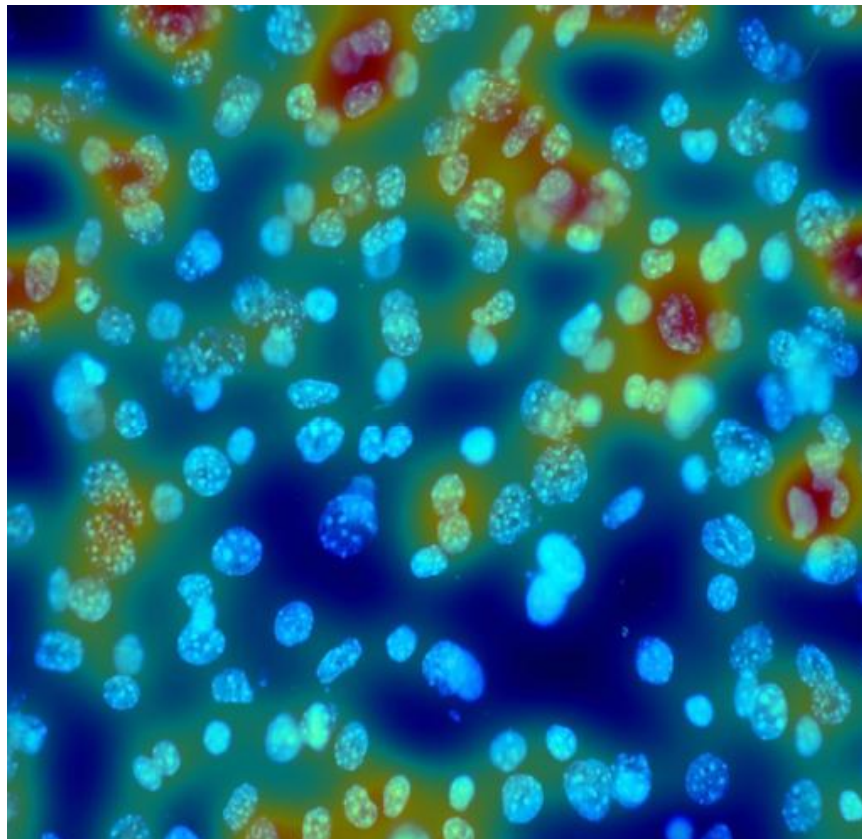


Figure 6. Circularity vs Area scatter plot. Green = Compact-Round, Blue = Elongated, Red = Irregular.

7.5 Density Heatmap Overlay

KDE spatial density heatmap overlaid on the ROI (55%/45% blend). Red zones indicate high cell concentration; blue = sparse. The distribution appears relatively uniform with slight clustering in the upper-centre region.

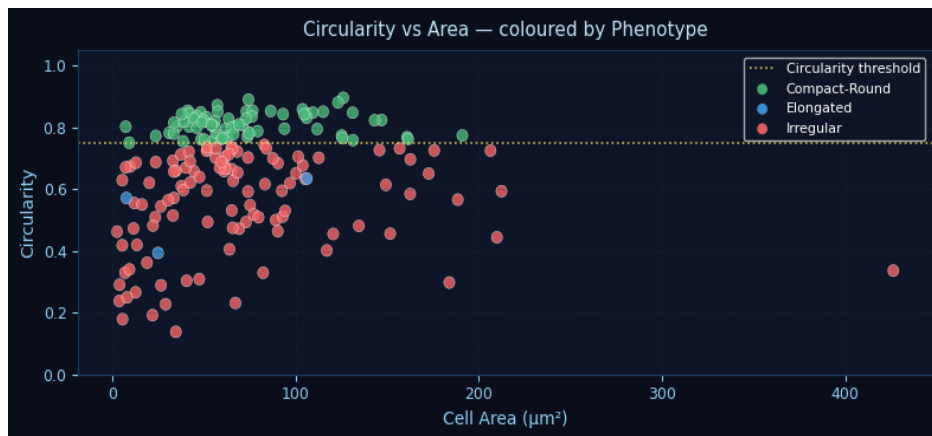


Figure 7. Density Heatmap Overlay. JET colormap: Red = dense, Blue = sparse.

7.6 Phenotype Summary

Rule-based morphological classification: Compact-Round (circularity > 0.75 AND solidity > 0.85), Elongated (aspect ratio > 1.8), Irregular (all others). In this sample: 71 Compact-Round (38.8%), 3 Elongated (1.6%), 109 Irregular (59.6%).

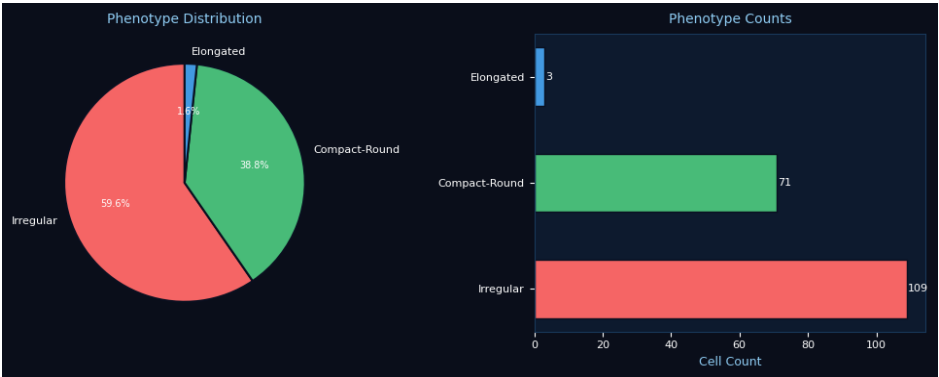


Figure 8. Phenotype Summary — pie chart (left) and bar chart (right) showing population breakdown.

7.7 Intensity Distribution

Per-cell mean fluorescence intensity distribution (ADU). 33rd percentile = 191, 66th percentile = 227, Mean = 203 ADU. The distribution is left-skewed with most cells showing high fluorescence (175–250 ADU), consistent with DAPI-stained nuclei.

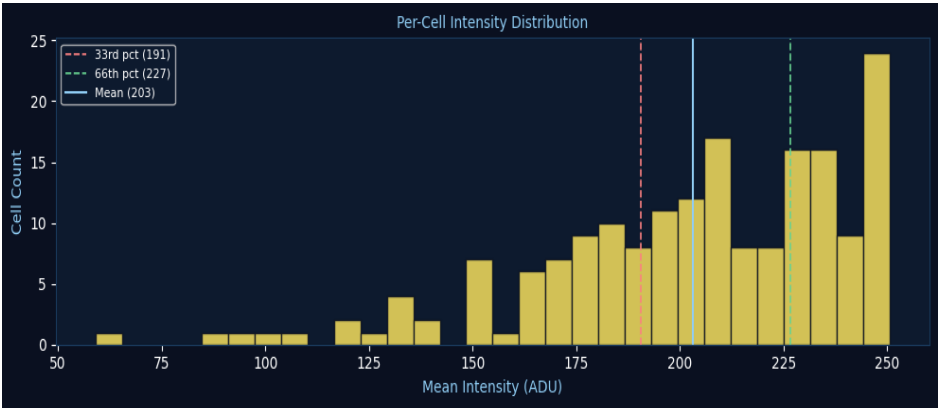


Figure 9. Per-Cell Intensity Distribution. Tier boundaries at 33rd and 66th percentiles separate Low/Mid/High subpopulations.

8. Output Statistics

Metric	Unit	Description
Total cell count	cells	Number of valid regions after all size and circularity filtering
Average cell area	µm²	Mean area calibrated by µm/pixel setting
Average circularity	0–1	Mean ($4\pi \cdot \text{area} / \text{perimeter}^2$); 1.0 = perfect circle
Average aspect ratio	ratio	Mean (width/height of bounding box); >2 = elongated

Metric	Unit	Description
Average solidity	0–1	Mean (contour area / convex hull area); 1.0 = fully compact
Cell density	cells/mm ²	Total cells / ROI area in mm ²
Confluence / Coverage	%	Fraction of ROI area covered by detected cell regions
Detection confidence	High/Med/Low	Heuristic from mask quality + contour-CCL count stability
Mean intensity	ADU	Average pixel brightness per cell (intensity feature)

9. Expert Features

Expert features are disabled by default and intended for advanced analyses beyond basic cell counting.

9.1 Spatial Density Heatmap

A KDE heatmap is generated from cell centroid coordinates and overlaid on the ROI. Quadrant analysis shows the count and percentage in each image quadrant, detecting uneven seeding or migration fronts.

9.2 Phenotypic Clustering

Rule-based morphological classification assigns each cell to Compact-Round, Elongated, or Irregular based on circularity, solidity, and aspect ratio. Results include pie/bar charts, per-phenotype statistics, and a color-annotated overlay.

9.3 Intensity Quantification

Per-cell mean intensity is measured using a contour mask. Cells are stratified into Low/Mid/High tiers at the 33rd and 66th percentiles. Useful for quantifying expression levels and identifying fluorescence subpopulations.

9.4 AI Interpretation (Google Gemini)

CellQuantX integrates with Google Gemini (model cascade: flash-lite → flash → flash-latest) for 6 analysis modes:

AI Mode	Description
Basic Summary	Population density, morphological health, growth phase estimate
Biological Hypothesis	Formal testable hypothesis on culture state (cycle, migration, differentiation)
Experimental Troubleshooting	Evaluates for fixation/osmotic issues; 2-3 corrective actions
Figure Legend Writer	Publication-ready legend in Nature Cell Biology style
Smart Outlier Detection	Flags extreme cells as real/artifact/clump with slider advice

AI Mode	Description
Clinical Grading	Grade I/II/III assessment with benign/borderline/malignant rationale

10. Export Options

Format	Contents
CSV	Per-cell data: Cell ID, Area ($\text{px}^2 + \mu\text{m}^2$), Circularity, Aspect Ratio, Solidity, optionally Mean Intensity and Phenotype
PNG	Annotated image with bounding boxes and ID labels (user-prompted filename)
PDF Report	Full report: summary metrics, AI interpretation, original + annotated images, all graphs, optional batch summary, first 50 rows cell data table

11. Quick Start & Workflow

- 9. Launch: streamlit run app.py — open <http://localhost:8501>
- 10. Upload one or more microscopy images (PNG, JPG, TIF, BMP, up to 200 MB)
- 11. Select a segmentation method or use a Quick Preset Profile
- 12. Set calibration: choose objective preset or enter $\mu\text{m}/\text{pixel}$ manually
- 13. Adjust Min Cell Area to exclude debris
- 14. Enable expert features (Heatmap, Phenotype, Intensity) as needed
- 15. Click Generate AI Interpretation for biological context
- 16. Export results: CSV, annotated PNG, or full PDF Report

Quick Preset Profiles

Preset	Method	Use Case
Brightfield Sparse	Global Threshold	Clear brightfield images with well-separated cells
Dense Clumps	Watershed	Dense cultures where cells touch or overlap
Uneven Illumination	Adaptive Threshold	Phase-contrast or images with shadows/gradients
Faint Cells	Enhanced Labeling Mode	Low-contrast or weakly stained cells on noisy background
Sedimentation / Clumped	Watershed	Settling cells in suspension; top-hat + heavy preprocessing

12. Troubleshooting

Problem	Solution
Cells merging into one region	Switch to Watershed; enable Separate touching cells; lower Watershed Strictness
Too many small debris counted	Increase Min Cell Area (e.g., 10 → 50 px ²); increase Min Circularity
Background counted as cells	Enable Invert Mask; switch to Adaptive Threshold; check flat-field correction
Faint cells not detected	Use Enhanced Labeling Mode; lower Min Cell Area to 5 px ²
AI Interpretation not working	Verify GEMINI_API_KEY in .env file; offline fallback activates automatically
PDF export fails	Ensure reportlab is installed: pip install reportlab
Very slow on large images	Enable Resize image before processing to use 512x512 ROI
Grid lines counted as cells	Enable Exclude counting chamber grid lines in Microscopy Parameters

13. Limitations & Future Work

13.1 Current Limitations

- Phenotypic clustering is rule-based (morphology only); no learned classification from labelled ground truth
- AI interpretation requires Gemini API key and internet; offline fallback is statistical only
- Z-stack TIFF files process only the first plane; volumetric counting not supported
- Only one channel is analyzed at a time; co-localization not implemented
- PDF report shows first 50 cells only in the data table

13.2 Planned Enhancements

17. Deep-learning segmentation — integrate Cellpose or StarDist for irregular overlapping cells
18. Metadata parsing — read pixel size from TIFF/OME-TIFF tags automatically
19. Viability counting — dual-channel fluorescence to differentiate live (green) and dead (red) cells
20. Z-stack support — 3D watershed segmentation for volumetric counting
21. Cell tracking — temporal analysis of cell migration across time-lapse frames
22. REST API mode — expose the pipeline for integration with lab automation systems